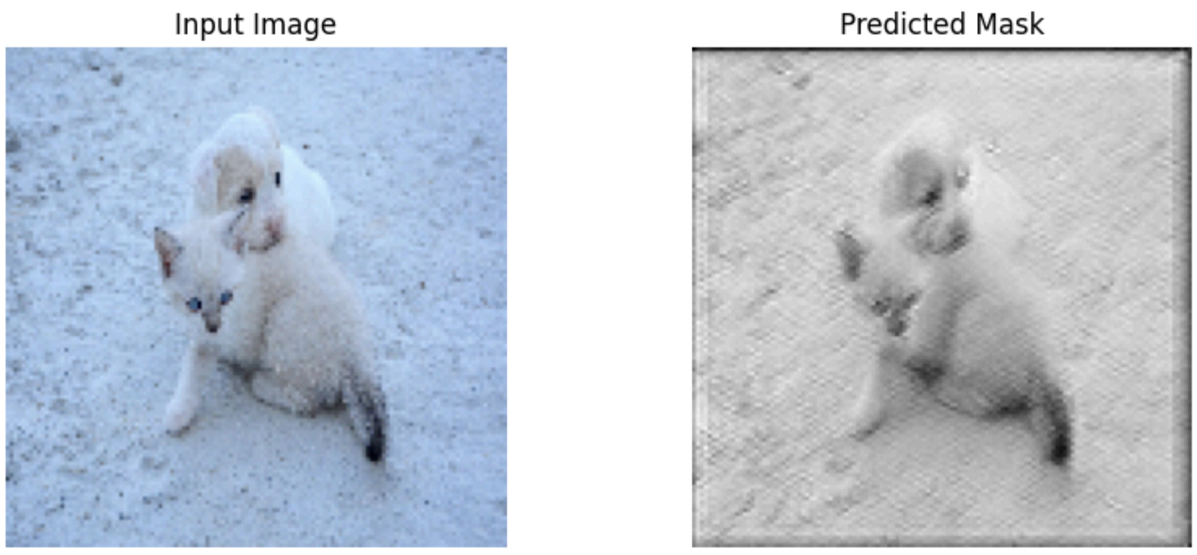
```
import tensorflow as tf
import numpy as np
import cv2
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
```

```
#  Load and Preprocess Image

img_path = "/content/jaclou-dl-dog-2445790_1920.jpg"

image = cv2.imread(img_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
image = cv2.resize(image, (128, 128))
image = image / 255.0

input_image = np.expand_dims(image, axis=0)


# 2. Define Small U-Net Model

def small_unet(input_size=(128,128,3)):
    inputs = layers.Input(input_size)

    # Encoder
    c1 = layers.Conv2D(16, (3,3), activation='relu', padding='same')(inputs)
    p1 = layers.MaxPooling2D((2,2))(c1)

    c2 = layers.Conv2D(32, (3,3), activation='relu', padding='same')(p1)
    p2 = layers.MaxPooling2D((2,2))(c2)

    # Bottleneck
    b = layers.Conv2D(64, (3,3), activation='relu', padding='same')(p2)

    # Decoder
    u1 = layers.UpSampling2D((2,2))(b)
    concat1 = layers.Concatenate()([u1, c2])
    c3 = layers.Conv2D(32, (3,3), activation='relu', padding='same')(concat1)

    u2 = layers.UpSampling2D((2,2))(c3)
    concat2 = layers.Concatenate()([u2, c1])
    c4 = layers.Conv2D(16, (3,3), activation='relu', padding='same')(concat2)

    outputs = layers.Conv2D(1, (1,1), activation='sigmoid')(c4)

    return models.Model(inputs, outputs)

model = small_unet()


# 3. Forward Pass

pred_mask = model.predict(input_image)


# 4. Visualization

plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(image)
plt.title("Input Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(pred_mask[0,:,:,0], cmap='gray')
plt.title("Predicted Mask")
plt.axis("off")

plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 222ms/step
```




```
import tensorflow as tf
from tensorflow.keras import layers, models

def unet(input_size=(128, 128, 3)):

    inputs = layers.Input(input_size)

    #  ENCODER (Downsampling)

    # Block 1
    c1 = layers.Conv2D(16, (3,3), activation='relu', padding='same')(inputs)
    c1 = layers.Conv2D(16, (3,3), activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D((2,2))(c1)

    # Block 2
    c2 = layers.Conv2D(32, (3,3), activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(32, (3,3), activation='relu', padding='same')(c2)
```

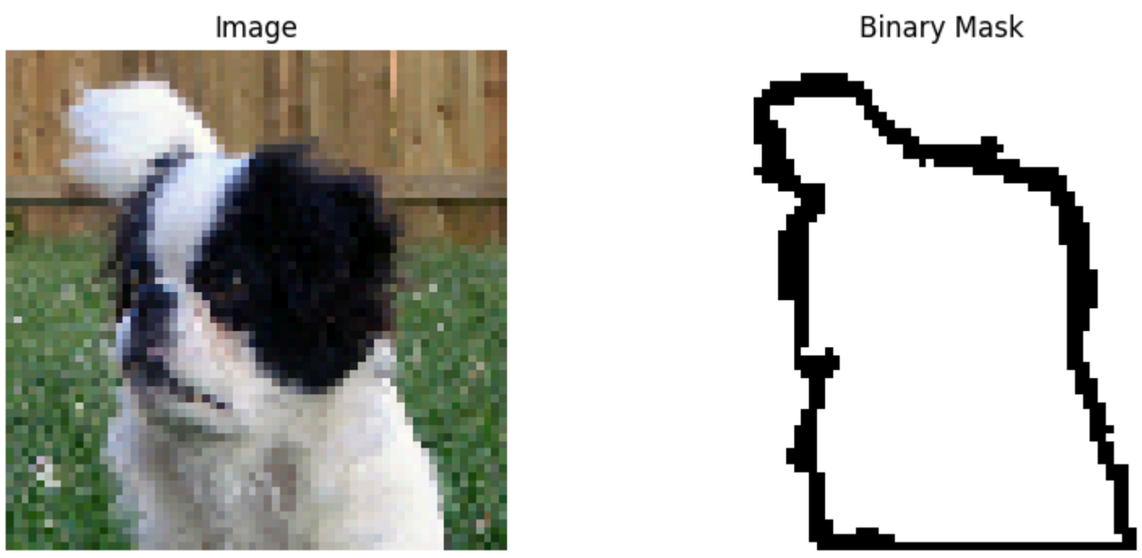
```
    c2 = layers.Conv2D(32, (3,3), activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D((2,2))(c2)


    #  BOTTLENECK

    b = layers.Conv2D(64, (3,3), activation='relu', padding='same')(p2)
    b = layers.Conv2D(64, (3,3), activation='relu', padding='same')(b)


    #  DECODER (Upsampling)

    # Up Block 1
    u1 = layers.UpSampling2D((2,2))(b)
    u1 = layers.Concatenate()([u1, c2])
    c3 = layers.Conv2D(32, (3,3), activation='relu', padding='same')(u1)
    c3 = layers.Conv2D(32, (3,3), activation='relu', padding='same')(c3)

    # Up Block 2
    u2 = layers.UpSampling2D((2,2))(c3)
    u2 = layers.Concatenate()([u2, c1])
    c4 = layers.Conv2D(16, (3,3), activation='relu', padding='same')(u2)
    c4 = layers.Conv2D(16, (3,3), activation='relu', padding='same')(c4)


    # OUTPUT LAYER


    outputs = layers.Conv2D(1, (1,1), activation='sigmoid')(c4)

    model = models.Model(inputs, outputs)

    return model


# Create and view model
model = unet()
model.summary()
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 128, 128, 3)	0	-
conv2d_15 (Conv2D)	(None, 128, 128, 16)	448	input_layer_2[0]…
conv2d_16 (Conv2D)	(None, 128, 128, 16)	2,320	conv2d_15[0][0]
max_pooling2d_4 (MaxPooling2D)	(None, 64, 64, 16)	0	conv2d_16[0][0]
conv2d_17 (Conv2D)	(None, 64, 64, 32)	4,640	max_pooling2d_4[…
conv2d_18 (Conv2D)	(None, 64, 64, 32)	9,248	conv2d_17[0][0]
max_pooling2d_5 (MaxPooling2D)	(None, 32, 32, 32)	0	conv2d_18[0][0]
conv2d_19 (Conv2D)	(None, 32, 32, 64)	18,496	max_pooling2d_5[…
conv2d_20 (Conv2D)	(None, 32, 32, 64)	36,928	conv2d_19[0][0]
up_sampling2d_2 (UpSampling2D)	(None, 64, 64, 64)	0	conv2d_20[0][0]
concatenate_4 (Concatenate)	(None, 64, 64, 96)	0	up_sampling2d_2[… conv2d_18[0][0]
conv2d_21 (Conv2D)	(None, 64, 64, 32)	27,680	concatenate_4[0]…
conv2d_22 (Conv2D)	(None, 64, 64, 32)	9,248	conv2d_21[0][0]
up_sampling2d_3 (UpSampling2D)	(None, 128, 128, 32)	0	conv2d_22[0][0]
concatenate_5 (Concatenate)	(None, 128, 128, 48)	0	up_sampling2d_3[… conv2d_16[0][0]
conv2d_23 (Conv2D)	(None, 128, 128, 16)	6,928	concatenate_5[0]…
conv2d_24 (Conv2D)	(None, 128, 128, 16)	2,320	conv2d_23[0][0]
conv2d_25 (Conv2D)	(None, 128, 128, 1)	17	conv2d_24[0][0]

Total params: 118,273 (462.00 KB)
Trainable params: 118,273 (462.00 KB)
Non-trainable params: 0 (0.00 B)

```
import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt


# 1️⃣ Load Dataset

dataset, info = tfds.load(
    "oxford_iiit_pet:4.0.0",
    with_info=True,
    as_supervised=False
)

train_data = dataset["train"]
test_data = dataset["test"]


# 2️⃣ Preprocessing Function
```

```
IMG_SIZE = 64
BATCH_SIZE = 8

def preprocess(sample):
    image = sample["image"]
    mask = sample["segmentation_mask"]

    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
    mask = tf.image.resize(
        mask,
        (IMG_SIZE, IMG_SIZE),
        method=tf.image.ResizeMethod.NEAREST_NEIGHBOR
    )

    image = tf.cast(image, tf.float32) / 255.0

    mask = tf.where(mask == 3, 0, 1)
    mask = tf.cast(mask, tf.float32)

    return image, mask


#  Apply Preprocessing

train_data = train_data.map(preprocess)
test_data = test_data.map(preprocess)


#  Shuffle & Batch

train_data = train_data.shuffle(1000).batch(BATCH_SIZE)
test_data = test_data.batch(BATCH_SIZE)


#  Visualize One Sample

for image, mask in train_data.take(1):
    plt.figure(figsize=(10,4))

    plt.subplot(1,2,1)
    plt.imshow(image[0])
    plt.title("Image")
    plt.axis("off")

    plt.subplot(1,2,2)
    plt.imshow(mask[0], cmap="gray")
    plt.title("Binary Mask")
    plt.axis("off")

    plt.show()
```




```
import tensorflow as tf
from tensorflow.keras import layers, models

def build_unet(input_shape=(64, 64, 3)):

    inputs = layers.Input(shape=input_shape)

    # Encoder
    c1 = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs)
    c1 = layers.Conv2D(32, 3, activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D()(c1)

    c2 = layers.Conv2D(64, 3, activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(64, 3, activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D()(c2)

    c3 = layers.Conv2D(128, 3, activation='relu', padding='same')(p2)
    c3 = layers.Conv2D(128, 3, activation='relu', padding='same')(c3)
    p3 = layers.MaxPooling2D()(c3)

    # Bottleneck
    b = layers.Conv2D(256, 3, activation='relu', padding='same')(p3)
    b = layers.Conv2D(256, 3, activation='relu', padding='same')(b)

    # Decoder
    u1 = layers.UpSampling2D()(b)
    u1 = layers.Concatenate()([u1, c3])
    c4 = layers.Conv2D(128, 3, activation='relu', padding='same')(u1)
    c4 = layers.Conv2D(128, 3, activation='relu', padding='same')(c4)

    u2 = layers.UpSampling2D()(c4)
    u2 = layers.Concatenate()([u2, c2])
    c5 = layers.Conv2D(64, 3, activation='relu', padding='same')(u2)
    c5 = layers.Conv2D(64, 3, activation='relu', padding='same')(c5)

    u3 = layers.UpSampling2D()(c5)
    u3 = layers.Concatenate()([u3, c1])
    c6 = layers.Conv2D(32, 3, activation='relu', padding='same')(u3)
    c6 = layers.Conv2D(32, 3, activation='relu', padding='same')(c6)

    outputs = layers.Conv2D(1, 1, activation='sigmoid')(c6)

    return models.Model(inputs, outputs)
```

```
model = build_unet()
model.summary()
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 64, 64, 3)	0	-
conv2d_41 (Conv2D)	(None, 64, 64, 32)	896	input_layer_4[0]…
conv2d_42 (Conv2D)	(None, 64, 64, 32)	9,248	conv2d_41[0][0]
max_pooling2d_9 (MaxPooling2D)	(None, 32, 32, 32)	0	conv2d_42[0][0]
conv2d_43 (Conv2D)	(None, 32, 32, 64)	18,496	max_pooling2d_9[…
conv2d_44 (Conv2D)	(None, 32, 32, 64)	36,928	conv2d_43[0][0]
max_pooling2d_10 (MaxPooling2D)	(None, 16, 16, 64)	0	conv2d_44[0][0]
conv2d_45 (Conv2D)	(None, 16, 16, 128)	73,856	max_pooling2d_10…
conv2d_46 (Conv2D)	(None, 16, 16, 128)	147,584	conv2d_45[0][0]
max_pooling2d_11 (MaxPooling2D)	(None, 8, 8, 128)	0	conv2d_46[0][0]
conv2d_47 (Conv2D)	(None, 8, 8, 256)	295,168	max_pooling2d_11…
conv2d_48 (Conv2D)	(None, 8, 8, 256)	590,080	conv2d_47[0][0]
up_sampling2d_7 (UpSampling2D)	(None, 16, 16, 256)	0	conv2d_48[0][0]
concatenate_9 (Concatenate)	(None, 16, 16, 384)	0	up_sampling2d_7[… conv2d_46[0][0]
conv2d_49 (Conv2D)	(None, 16, 16, 128)	442,496	concatenate_9[0]…
conv2d_50 (Conv2D)	(None, 16, 16, 128)	147,584	conv2d_49[0][0]
up_sampling2d_8 (UpSampling2D)	(None, 32, 32, 128)	0	conv2d_50[0][0]
concatenate_10 (Concatenate)	(None, 32, 32, 192)	0	up_sampling2d_8[… conv2d_44[0][0]
conv2d_51 (Conv2D)	(None, 32, 32, 64)	110,656	concatenate_10[0…
conv2d_52 (Conv2D)	(None, 32, 32, 64)	36,928	conv2d_51[0][0]
up_sampling2d_9 (UpSampling2D)	(None, 64, 64, 64)	0	conv2d_52[0][0]
concatenate_11 (Concatenate)	(None, 64, 64, 96)	0	up_sampling2d_9[… conv2d_42[0][0]
conv2d_53 (Conv2D)	(None, 64, 64, 32)	27,680	concatenate_11[0…
conv2d_54 (Conv2D)	(None, 64, 64, 32)	9,248	conv2d_53[0][0]
conv2d_55 (Conv2D)	(None, 64, 64, 1)	33	conv2d_54[0][0]

Total params: 1,946,881 (7.43 MB)
Trainable params: 1,946,881 (7.43 MB)
Non-trainable params: 0 (0.00 B)

```
# Compile Model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

EPOCHS = 3

history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=EPOCHS
)
```

```
Epoch 1/3
460/460 ━━━━━━━━━━━━━━━━━━━━ 606s 1s/step - accuracy: 0.8876 - loss: 0.3243 - val_accuracy: 0.8863 - val_loss: 0.2846
Epoch 2/3
460/460 ━━━━━━━━━━━━━━━━━━━━ 556s 1s/step - accuracy: 0.8934 - loss: 0.2662 - val_accuracy: 0.8951 - val_loss: 0.2586
Epoch 3/3
460/460 ━━━━━━━━━━━━━━━━━━━━ 554s 1s/step - accuracy: 0.8999 - loss: 0.2459 - val_accuracy: 0.9019 - val_loss: 0.2422
```

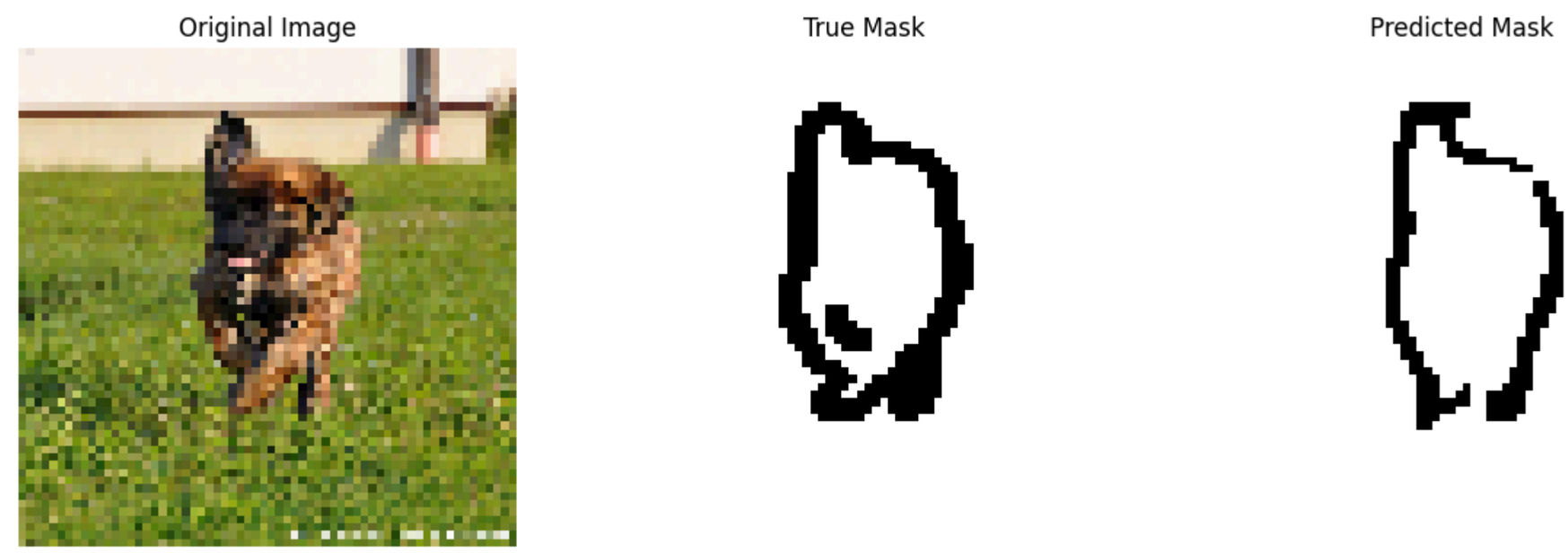
```
import tensorflow as tf

def iou_metric(y_true, y_pred):
    y_pred = tf.cast(y_pred > 0.5, tf.float32)

    intersection = tf.reduce_sum(y_true * y_pred)
    union = tf.reduce_sum(y_true) + tf.reduce_sum(y_pred) - intersection

    return intersection / (union + 1e-7)
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', iou_metric]
)
```

```
results = model.evaluate(test_data)

print("Test Loss:", results[0])
print("Test Accuracy:", results[1])
print("Test IoU:", results[2])
```

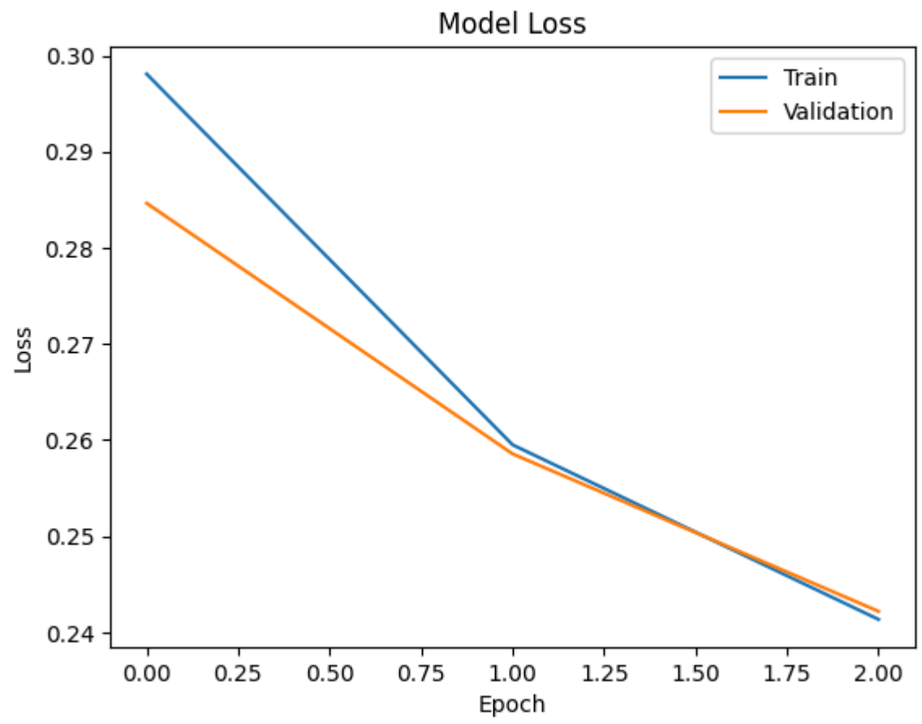
```
459/459 ━━━━━━━━━━━━━━━━━━━━ 121s 262ms/step - accuracy: 0.9014 - iou_metric: 0.8967 - loss: 0.2436
Test Loss: 0.24217237532138824
Test Accuracy: 0.9018575549125671
Test IoU: 0.8972008228302002
```

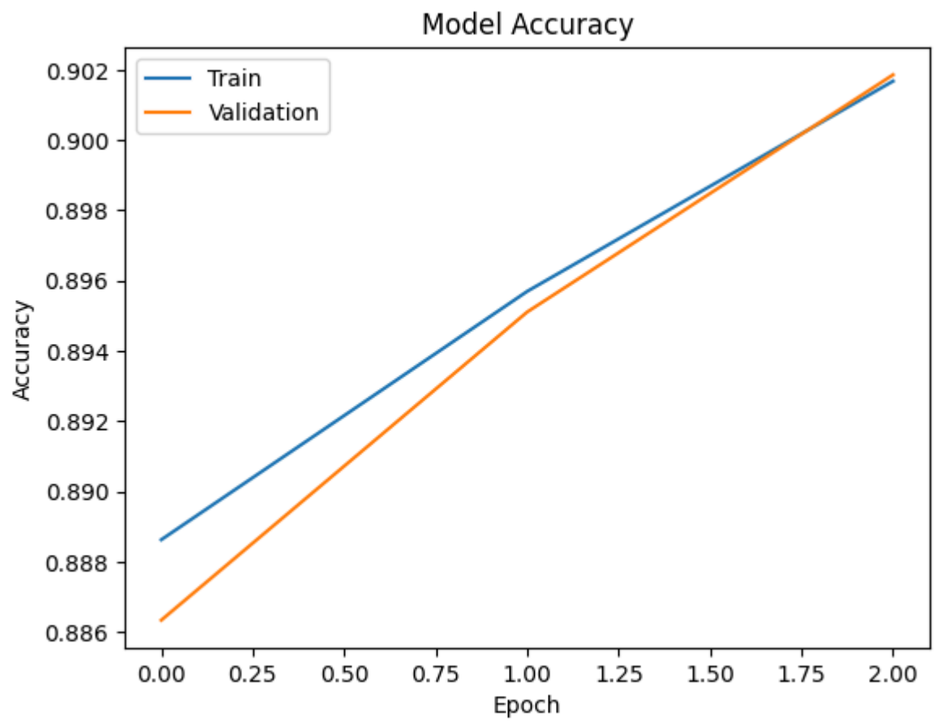
```
import matplotlib.pyplot as plt
import numpy as np

for images, masks in test_data.take(1):

    preds = model.predict(images)

    # Convert tensors to numpy
    img = images[0].numpy()
    true_mask = masks[0].numpy().squeeze()
    pred_mask = (preds[0] > 0.5).astype(np.uint8).squeeze()

    plt.figure(figsize=(15,5))

    # Original Image
    plt.subplot(1,3,1)
    plt.imshow(img)
    plt.title("Original Image")
    plt.axis("off")

    # True Mask
    plt.subplot(1,3,2)
    plt.imshow(true_mask, cmap="gray", vmin=0, vmax=1)
    plt.title("True Mask")
    plt.axis("off")

    # Predicted Mask
    plt.subplot(1,3,3)
    plt.imshow(pred_mask, cmap="gray", vmin=0, vmax=1)
    plt.title("Predicted Mask")
    plt.axis("off")

    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 667ms/step
```




```
import matplotlib.pyplot as plt

# Loss Plot
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()

# Accuracy Plot
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',

)

results = model.evaluate(test_data)

print("Test Loss:", results[0])
print("Test Accuracy:", results[1])
```






```
432/459 ━━━━━━━━━━━━━━━━━━━━ 7s 282ms/step - accuracy: 0.9014 - iou_metric: 0.8967 - loss: 0.2436
```

summary:-

In this lab, we implemented the U-Net architecture for semantic image segmentation using TensorFlow. Initially, a small U-Net model was tested on a single image to understand how segmentation masks are generated at the pixel level. After that, the Oxford-IIIT Pet dataset was loaded and preprocessed by resizing images, normalizing pixel values, and converting segmentation masks into binary format.

A full U-Net model with encoder–decoder structure and skip connections was then built and trained for a few epochs. The model was compiled using the Adam optimizer and binary crossentropy loss, and evaluated using accuracy and IoU metrics. The results demonstrated that U-Net effectively performs precise image segmentation, achieving good accuracy and overlap between predicted and true masks.